IMPERIAL COLLEGE LONDON

DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING
EXAMINATIONS 2018

EEE/EIE PART III/IV: MEng, BEng and ACGI

Corrected copy

ARTIFICIAL INTELLIGENCE

Tuesday, 4 December 9:00 am

Time allowed: 3:00 hours

There are FIVE questions on this paper.

Answer FOUR questions.

All questions carry equal marks

Any special instructions for invigilators and information for candidates are on page 1.

Examiners responsible First Marker(s) : J.V. Pitt
Second Marker(s) : T-K. Kim

© Imperial College London

3-16. ARTIFICIAL INTELLIGENCE

1. One way for writing Regular Expressions uses a single character as itself, uses the plus symbol (+) to indicate that there should be one or more instances of the preceding character, and the zero symbol (0) to indicate that there should be zero or more instances of the preceding character.

   So, for example, the Regular Expression a^+b^0c defines the language of strings consisting of one or more as, followed by zero or more bs, followed by a single c.

   A state machine, with identified start and end states, can be used to test if a string matches the Regular Expression. For example, Figure 1.1 shows the state machine for the Regular Expression a^+b^0c.

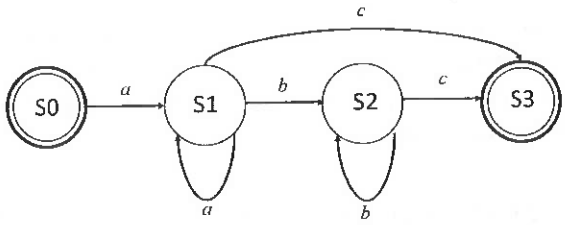


Figure 1.1 State machine for the Regular Expression a^+b^0c

This question explores how Prolog can be used to test and generate strings matching a regular expression.

a) Define a Prolog representation for the string of characters in a regular expression. The end of the expression should be explicitly identified.

[2]

b) Define a predicate `transition( Scurr, Snext, Ch )` which encodes the transitions of the state machine shown in Figure 1.1.

Note that `transition( Scurr, Snext, Ch )` should be true if there is a transition from current state `Scurr` to next state `Snext` when the next character in the string is `Ch`.

[2]

c) The Prolog predicate `match( RE )` should be true when `RE` represents a string which matches the Regular Expression a^+b^0c. Define two versions of `match/1`, one using cut, and one without using cut. Explain the difference between the two versions.

[6]

d) Give an example of a query when `RE` is instantiated with a value representing a string. Explain what happens if `RE` is not instantiated.

[2]

e) Therefore, instead of using `match/1` 'backwards' to generate strings, define a Prolog predicate `generate( RE )` which generates strings matching the Regular Expression a^+b^0c. Explain the approach being used to generate strings, and comment briefly on its output.

[8]

 ©Imperial College London

2. **The Valueneutralese Family River-Crossing Problem.** The Valueneutralese Family are on a family day out with some friends. There is the mom, dad, 2 daughters, 2 sons, and (for reasons this question is never going to make entirely clear) a policeman and a thief. While they are out walking, they come across a river. By the side of the river, there is a small boat, and what appears to be the remnants of three missionaries and a goat. Ignoring these, the problem facing the Valueneutralese Family is to find a way of crossing the river using the boat, subject to the following constraints:

- The boat can carry no more than 2 people (either adults or children);
- Only the responsible adults (mom, dad, policeman) can row the boat;
- The dad can not be alone on a bank with either daughter without their mom;
- The mom can not be alone on a bank with either son without their dad; and
- The thief can not be alone with any of the family without the policeman.

You are given a predicate `pick/3` and you are told that `pick( L1, L2, L3 )` will generate (on backtracking) all singleton members and pairs of members contained in list `L1`, and return these as `L2`, while all the remaining members of the list are returned as `L3`.

a) Formulate the state space so that it could be used with the General Graph Search Engine (GGSE) to find a solution to the Valueneutralese Family River-Crossing Problem. (Using `pick/3` without further definition is allowed; an actual solution to the problem is not required.)

[10]

b) In the context of the General Graph Search Engine, and using the state space formulation from part (a) as an example, explain the terms *state*, *node*, *path* and *graph*.

[4]

c) Briefly explain the difference between *breadth first*, *depth first*, *uniform cost* and *best first* search algorithms.

Using appropriate evaluation criteria, compare and contrast the four algorithms.

Briefly comment on the suitability of each algorithm for solving the Valueneutralese Family River-Crossing Problem.

[6]

3. a) Given an implicit definition of a graph G' as a 2-tuple $G' = \langle S, Op \rangle$, explain what is S, and what is Op.

Show how this definition can be used to generate all the paths in a graph G'.

Given an explicit definition of a graph G a 3-tuple $G = \langle N, E, R \rangle$, what conditions need to be satisfied for the paths in G' to be the same as the paths in G.

[4]

b) Prove that the A^* search algorithm is optimal.

[3]

c) Explain the difference between the *minimax*, *minimax to fixed ply*, and *alpha-beta* algorithms for playing 2-player games.

[3]

d) Briefly explain how the *minimax* algorithm could be used, and if necessary adapted, for 3-player games, in which it is possible for a move by one player to eliminate another, and the last player is the winner.

[3]

e) Define a Kripke Model, and explain how it is used to give a semantics for formulas of Modal Logic.

[3]

f) Using the Calculus KE for Modal Logic, prove that the following formulas are valid in the logic **S5** (KE-trees (proofs) should be properly annotated to show the inference steps, i.e. showing the inference rule used, line number of the major and minor premises, and label for the world):

(*i*) $\Box P \rightarrow P$
(*ii*) $P \rightarrow \Box \Diamond P$
(*iii*) $\Box P \rightarrow \Box \Box P$

Explain why these proofs are to be expected.

[4]

4. a) Explain the Prolog proof search procedure, and how it uses *resolution* and *unification*.

[4]

b) `retract` is a Prolog built-in meta-predicate which is used to remove facts or rules from the database while a program is executing.

Consider the following two Prolog programs:

```
p :- q, t.
p :- q, r, s.
q.
r :- retract( q ).
s :- fail.
t.
```

```
p :- q, r, s.
p :- q, t.
q.
r :- retract( q ).
s :- fail.
t.
```

Explain the result of each program in response to the query: `?- p.`

Using this, remark on the *soundness* and *completeness* of the Prolog proof search procedure.

[4]

c) Consider the following Prolog program:

```
likesCats( X ) :-
        \+ likesDogs( X ).

likesDogs( bob ).
```

Explain the Prolog output in response to each of the following queries:

(*i*) `?- likesCats( bob ).`
(*ii*) `?- likesCats( jeremy ).`
(*iii*) `?- likesCats( X ).`

[3]

d) Consider the following English statements:

Every hunter is insecure.
Every lion is a threat.
Everyone who is insecure hunts anyone who is threat.
Everyone who hunts and shoots anyone else, kills them.
Walter is a hunter.
Cecil is a lion.
Walter shoots Cecil.

Express these statements in first-order predicate logic, and convert them to a set of Horn clauses.

[4]

e) Prove, using resolution and proof by negation, that *Walter kills Cecil*. The proof should show all the unifiers.

[5]

5. In answering this question, it is essential that any KE-trees (proofs) are properly annotated to show the inference steps (i.e. showing the inference rule used, line number of the major and minor premises).

a) Explain the KE proof search procedure, and remark on its *soundness* and *completeness*.

[4]

b) Using the Calculus KE, show that:

$$\{s \rightarrow p, w \wedge \neg z, \neg p, \neg z \rightarrow (s \vee q \vee r), (w \vee y) \rightarrow \neg q\} \models r$$

[2]

c) Removing the formula $\neg p$ from the set of premises in part (a), use the Calculus KE to show that:

$$\{s \rightarrow p, w \wedge \neg z, \neg z \rightarrow (s \vee q \vee r), (w \vee y) \rightarrow \neg q\} \not\models r$$

State which valuation(s) make the premises true and the conclusion false.

[3]

d) Using the Calculus KE, disprove:

$$\{p \vee \neg q \vee r, \neg q \rightarrow \neg p, r \rightarrow (\neg p \vee q)\} \models r$$

State which valuation(s) make the premises true and the conclusion false.

[3]

e) In the *Obligatio Game*, a finite number n of rounds is chosen, depending on the severity of the exam. The examiner then gives the candidate successive assertions, $\phi_1, \phi_2, \ldots, \phi_n$ that she has to either *accept* or *reject* as each one is put forward. In the former case, ϕ_i is added to the candidate's stock of commitments; in the latter case, the negation $\neg\phi_i$ is added.

The candidate passes the exam if she maintains the consistency of her stock of commitments throughout all n assertions.

Suppose the candidate is exposed (successively) to the following three assertions:

1. $p \rightarrow \neg(q \vee r)$
2. $p \wedge q$
3. $q \leftrightarrow r$

Using the KE calculus, show what sequences of answers would ensure that the student passes the exam. (Hint: use KE for model building, for all combinations of *accept* or *reject* of the assertions.)

[8]